UNIVERSIDAD AUTÓNOMA DE CHIRIQUÍ
FACULTAD DE ECONOMÍA — ESCUELA DE CIENCIAS COMPUTACIONALES
Licenciatura en Gestión de Tecnología de la Información

GUÍA DE LABORATORIO N.° 1

Preparación del Entorno de Desarrollo

Asignatura	Programación V (PROG 309)
Eje temático	N.° 1 — Introducción a los Patrones Arquitecturales
Semana	Semana 1
Docente	Alexis Flores Franco
Modalidad	Presencial — Laboratorio de Cómputo
Duración estimada	2 sesiones de laboratorio (Jueves y Viernes, 1 hora con 35 minutos cada una)
Software requerido	JDK 17 (Java 17 LTS), Eclipse IDE, Git, herramienta de diagramación (ej. draw.io)
Ponderación	Forma parte del 35% de Tareas, investigaciones, talleres y otras

1. Introducción

Antes de estudiar en profundidad cada patrón de arquitectura de software, es indispensable contar con un entorno de trabajo estable, ordenado y consistente durante todo el semestre. Este primer laboratorio no está centrado en un patrón arquitectónico en particular, sino en preparar las herramientas de desarrollo (IDE, lenguaje de programación y control de versiones) que se utilizarán en todos los proyectos del curso, además de dar un primer acercamiento visual al patrón de capas mediante la observación de una aplicación cotidiana.

Un entorno de desarrollo bien configurado desde el inicio evita pérdidas de tiempo posteriores y permite que el estudiante concentre su esfuerzo en comprender la arquitectura de las aplicaciones, y no en resolver problemas de instalación a mitad del semestre.

2. Marco Teórico

2.1 El entorno de desarrollo integrado (IDE)

Un IDE (Integrated Development Environment) es una aplicación que reúne, en un solo lugar, el editor de código, el depurador (debugger), el compilador o intérprete, y herramientas de gestión de proyectos. Ejemplos ampliamente usados en la industria son Visual Studio Code, IntelliJ IDEA y Eclipse.

En este curso se utilizará Eclipse IDE (específicamente la edición “Eclipse IDE for Enterprise Java and Web Developers”) como entorno estándar para todos los laboratorios del semestre. Eclipse ofrece integración nativa con Maven, con servidores de aplicaciones (Tomcat) y con Git, lo cual será especialmente útil en los ejes temáticos de patrón de capas y patrón cliente-servidor.

2.2 Control de versiones con Git

Git es un sistema de control de versiones distribuido que permite registrar el historial de cambios de un proyecto, trabajar de forma colaborativa y recuperar versiones anteriores del código cuando sea necesario. Un repositorio (repo) es el espacio donde Git almacena ese historial; puede alojarse de forma local o remota, por ejemplo en GitHub.

2.3 El patrón de capas como primer acercamiento

El patrón de capas organiza un sistema en niveles jerárquicos, donde cada capa tiene una responsabilidad específica y solo se comunica con las capas adyacentes. Las tres capas más comunes son:

- Capa de presentación: lo que el usuario ve e interactúa (interfaz gráfica).
- Capa de negocio o lógica: las reglas, cálculos y procesos propios de la aplicación.
- Capa de acceso a datos: la comunicación con la base de datos u otra fuente de información.

En este laboratorio no se implementará el patrón de capas todavía —eso ocurrirá en el Eje Temático N.° 2—; el objetivo es únicamente reconocerlo visualmente en una aplicación conocida.

3. Objetivos

3.1 Objetivo general

Configurar el entorno de desarrollo que se utilizará durante el semestre y realizar un primer acercamiento visual al patrón de capas.

3.2 Objetivos específicos

- Instalar y configurar el JDK 17 y Eclipse IDE para el desarrollo en Java durante el semestre.
- Crear un proyecto Maven base con estructura de paquetes estilo MVC (modelo, vista, controlador).
- Instalar y configurar Git como sistema de control de versiones.
- Crear y preparar un repositorio de trabajo personal (local y remoto en GitHub) para las prácticas del semestre.
- Reconocer visualmente las capas de una aplicación cotidiana y representarlas en un diagrama.
- Documentar el procedimiento realizado mediante un informe técnico de laboratorio.

4. Competencias a Desarrollar

- Reconoce e infiere sobre la importancia de los patrones de diseño arquitecturales.
- Capacidad de aplicar los conocimientos en la práctica.
- Capacidad para organizar y planificar el tiempo.

5. Materiales y Recursos

Equipo	Computadora del laboratorio de cómputo, con acceso a internet
IDE	Eclipse IDE for Enterprise Java and Web Developers
Lenguaje / SDK	Java 17 LTS (JDK 17, distribución Eclipse Temurin/Adoptium recomendada)
Control de versiones	Git y una cuenta en GitHub (o plataforma equivalente)
Diagramación	Herramienta de diagramación en línea (ej. draw.io / diagrams.net)
Documento	Plantilla de Informe de Laboratorio (ver planificación didáctica de la asignatura)

6. Procedimiento

Actividad 1 — Instalación de JDK 17 y Eclipse IDE

Paso A. Instalar el JDK 17 (Java 17 LTS) y verificarlo

1. Descargar el JDK 17 LTS desde el sitio oficial de Eclipse Temurin/Adoptium (adoptium.net) y ejecutar el instalador con las opciones por defecto.
2. Abrir una terminal (Símbolo del sistema en Windows, Terminal en macOS/Linux) y verificar la instalación con los comandos java -version y javac -version; ambos deben mostrar la versión 17.x.
3. Si el comando no es reconocido, configurar las variables de entorno: en Windows, agregar JAVA_HOME y añadir %JAVA_HOME%\bin al Path; en macOS/Linux, agregar la exportación correspondiente en el archivo de perfil de la shell (.bashrc, .zshrc, etc.).

Paso B. Instalar Eclipse IDE

4. Descargar el instalador "Eclipse Installer" desde el sitio oficial (eclipse.org/downloads).
5. En el instalador, seleccionar el paquete "Eclipse IDE for Enterprise Java and Web Developers" (incluye soporte para Maven y servidores de aplicaciones que se usarán más adelante en el curso).
6. Completar la instalación y abrir Eclipse por primera vez, seleccionando o creando el workspace (espacio de trabajo) donde se guardarán los proyectos del semestre.
7. Verificar que Eclipse reconozca el JDK 17 instalado: Window > Preferences > Java > Installed JREs; si no aparece, agregarlo manualmente con Add... > Standard VM y ubicar la carpeta de instalación del JDK.

Paso C. Crear el proyecto base con estructura MVC

8. En Eclipse: File > New > Maven Project > marcar "Create a simple project" o seleccionar el arquetipo maven-archetype-quickstart.
9. Definir Group Id: com.unachi.prog5, Artifact Id: prog5-base y Version: 1.0.0, y finalizar la creación del proyecto.
10. Verificar que el proyecto use JDK 17: clic derecho sobre el proyecto > Properties > Java Build Path > pestaña Libraries, y confirmar que la JRE System Library sea JavaSE-17 (si no lo es, editarla y seleccionar la versión correcta).
11. Dentro de src/main/java, crear los siguientes paquetes: com.unachi.prog5.model, com.unachi.prog5.view y com.unachi.prog5.controller.

Paso D. Crear las clases base y ejecutar el proyecto

Crear las siguientes cuatro clases con el contenido indicado (este pequeño proyecto ya adelanta, de forma muy simple, la separación en modelo/vista/controlador que se estudiará a fondo en el Eje Temático N.° 4):

com.unachi.prog5.model.Student

```
package com.unachi.prog5.model;

public class Student {
    private final String id;
    private final String name;

    public Student(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String summary() {
        return "Student{id='" + id + "', name='" + name + "'}";
    }
}
```

com.unachi.prog5.view.ConsoleView

```
package com.unachi.prog5.view;

public class ConsoleView {
    public void showMessage(String msg) {
        System.out.println(msg);
    }
}
```

com.unachi.prog5.controller.AppController

```
package com.unachi.prog5.controller;

import com.unachi.prog5.model.Student;
import com.unachi.prog5.view.ConsoleView;

public class AppController {
    private final ConsoleView view;

    public AppController(ConsoleView view) {
        this.view = view;
    }

    public void run() {
        Student s = new Student("2026-001", "Nombre Apellido");
        view.showMessage("Hola Programacion V");
        view.showMessage("Modelo: " + s.summary());
    }
}
```

com.unachi.prog5.Main

```
package com.unachi.prog5;

import com.unachi.prog5.controller.AppController;
import com.unachi.prog5.view.ConsoleView;

public class Main {
    public static void main(String[] args) {
        AppController controller = new AppController(new ConsoleView());
        controller.run();
    }
}
```

12. Guardar todos los archivos y ejecutar Main.java con clic derecho > Run As > Java Application.
13. Verificar en la consola de Eclipse que se muestre la salida: “Hola Programacion V” seguido de “Modelo: Student{id='2026-001', name='Nombre Apellido'}”.

Actividad 2 — Instalación y configuración de Git

14. Descargar e instalar Git desde su sitio oficial (git-scm.com), si no está ya instalado en el equipo. En Windows, dejar marcadas las opciones por defecto del instalador (incluye Git Bash).
15. Abrir una terminal (o la terminal integrada de Eclipse: Window > Show View > Other... > Terminal) y verificar la instalación con el comando git --version; debe mostrar algo similar a git version 2.4x.x.
16. Configurar el nombre de usuario: git config --global user.name "Tu Nombre".
17. Configurar el correo electrónico: git config --global user.email "tu_correo@ejemplo.com" (usar preferiblemente el correo con el que se registrará en GitHub).
18. Verificar que la configuración quedó guardada correctamente con el comando git config --list; debe mostrar las líneas user.name y user.email con los datos ingresados.
19. Crear una cuenta en GitHub (github.com) si el estudiante no cuenta ya con una, utilizando preferiblemente su correo institucional.

Actividad 3 — Creación y preparación del repositorio de trabajo

Paso A. Crear el repositorio remoto en GitHub

20. Iniciar sesión en GitHub y hacer clic en el botón New (o el símbolo + > New repository).
21. Nombrar el repositorio programacion-v-2026 y seleccionar Public o Private, según lo indique el docente.

22. IMPORTANTE: no marcar la opción “Add a README file” ni ninguna otra opción de inicialización, para evitar conflictos al conectar el repositorio local. Hacer clic en Create repository.
23. En la página del repositorio recién creado, copiar la URL de HTTPS (botón verde Code > HTTPS), similar a: https://github.com/tu-usuario/programacion-v-2026.git

Paso B. Inicializar el repositorio local

24. Abrir una terminal dentro de la carpeta de trabajo ProgramacionV_2026-II creada en la Actividad 1 (en Eclipse también se puede abrir con clic derecho sobre el proyecto > Show In > Terminal).
25. Inicializar Git en la carpeta con el comando git init; debe mostrar el mensaje Initialized empty Git repository in .../.git/
26. Verificar el estado del repositorio con git status; debe indicar que se está en la rama principal (main o master) y que aún no hay commits (“No commits yet”).

Paso C. Crear el primer commit

27. Crear, dentro de la carpeta de trabajo, un archivo README.md que incluya: nombre completo del estudiante, nombre de la asignatura (Programación V) y semestre (2026-II). Puede crearse desde Eclipse (File > New > File) o desde la terminal.
28. Agregar el archivo al área de preparación (staging) con el comando git add README.md (o git add . para agregar todos los archivos nuevos del proyecto).
29. Verificar qué quedó preparado para el commit con git status; el archivo debe aparecer listo para confirmarse (en verde).
30. Realizar el primer commit del semestre con el comando: git commit -m "Primer commit - configuracion inicial del entorno"

Paso D. Conectar y subir el repositorio local al remoto

31. Conectar el repositorio local con el repositorio remoto de GitHub: git remote add origin https://github.com/tu-usuario/programacion-v-2026.git (usar la URL copiada en el Paso A).
32. Verificar que el remoto quedó correctamente conectado con git remote -v; debe mostrar origin listado tanto para fetch como para push, con la URL del repositorio.
33. Asegurar que la rama principal se llame main con el comando: git branch -M main
34. Subir el commit al repositorio remoto con: git push -u origin main (la opción -u deja configurado el enlace entre la rama local y la remota, para que en adelante baste con escribir git push).
35. Recargar la página del repositorio en GitHub y verificar que el archivo README.md y el commit realizado aparezcan reflejados en línea.

Paso E (opcional). Configurar un archivo .gitignore para proyectos Java/Maven/Eclipse

Para evitar subir al repositorio archivos generados automáticamente por Eclipse y Maven, crear un archivo .gitignore en la raíz del proyecto con el siguiente contenido, y repetir los pasos de add, commit y push:

```
target/
*.class
.settings/
.project
.classpath
.metadata/
```

Actividad 4 — Reconocimiento visual del patrón de capas

36. Elegir una aplicación de uso cotidiano en el celular o la computadora (por ejemplo, una app bancaria, una red social o una tienda en línea).
37. Identificar qué parte de la aplicación corresponde a la capa de presentación (lo que se ve y con lo que se interactúa).
38. Inferir qué procesos podrían pertenecer a la capa de negocio (por ejemplo, el cálculo de un total de compra o la validación de una contraseña).
39. Inferir qué información probablemente se almacena y se consulta en la capa de datos (por ejemplo, el historial de pedidos o la lista de contactos).
40. Elaborar, en la herramienta de diagramación, un esquema simple de tres niveles (presentación, negocio, datos) que represente la aplicación elegida.

Actividad 5 — Elaboración del informe de laboratorio

41. Redactar un informe breve utilizando la Plantilla de Informe de Laboratorio de la asignatura, describiendo el proceso de configuración del entorno de desarrollo.
42. Incluir en el informe una captura de pantalla del primer commit realizado en GitHub y el diagrama de capas elaborado en la Actividad 4.
43. Subir el informe y el diagrama al repositorio de trabajo, dentro de una carpeta llamada laboratorio1/.

7. Resultado Esperado

Al finalizar el laboratorio, el estudiante debe contar con:

- JDK 17 y Eclipse IDE instalados y funcionales, con el proyecto Maven prog5-base reconociendo correctamente JavaSE-17.
- Un proyecto con estructura de paquetes MVC (model, view, controller) que compila y ejecuta correctamente, mostrando la salida esperada en consola.
- Git instalado y configurado, con un repositorio remoto en GitHub conectado al proyecto local.
- Al menos un commit visible en el historial del repositorio, subido correctamente al remoto (git push).
- Un diagrama de tres capas (presentación, negocio, datos) de una aplicación cotidiana.
- Un informe de laboratorio completo, con evidencias, subido al repositorio de trabajo.

8. Preguntas de Análisis

Responder de forma individual, con base en lo realizado durante la práctica:

1. ¿Por qué es importante llevar un control de versiones (Git) desde el primer proyecto del semestre y no solo en proyectos grandes?
2. En la aplicación que eligió para el diagrama de capas, ¿qué problema podría ocurrir si la capa de presentación accediera directamente a la base de datos, sin pasar por la capa de negocio?
3. ¿Qué ventajas ofrece trabajar con un repositorio remoto (como GitHub) en lugar de guardar el código únicamente en la computadora local?
4. Investigue: ¿qué diferencia existe entre un IDE y un simple editor de texto como Notepad++ o Sublime Text?

9. Rúbrica de Evaluación

Criterio	Puntaje	Descripción
Instalación de JDK 17 y Eclipse	10%	Instala correctamente el JDK 17 y Eclipse IDE, y verifica la versión desde la terminal.
Proyecto Maven con estructura MVC	15%	Crea el proyecto Maven con los paquetes model/view/controller y ejecuta Main con la salida esperada.
Configuración de Git y GitHub	15%	Instala y configura Git, crea el repositorio remoto y conecta correctamente el repositorio local.
Primer commit y push	10%	Realiza correctamente el primer commit y lo sube al repositorio remoto (git push).
Diagrama de capas	20%	Identifica correctamente las tres capas en la aplicación elegida y las representa con claridad.

Informe de laboratorio	20%	Entrega el informe completo, con evidencias y siguiendo la plantilla institucional.
Preguntas de análisis	10%	Responde las preguntas con argumentos claros y relacionados con la teoría vista en clase.

10. Conclusiones (a completar por el estudiante)

Redacte de tres a cinco líneas explicando lo aprendido durante esta práctica y su relación con los conceptos teóricos vistos en la Semana 1.

---

11. Referencias

Huaman, W. C. (2018). Los 10 patrones comunes de arquitectura de software. Medium.

Blancarte, O. (2020). Introducción a la arquitectura de software.

Documentación oficial de Git: https://git-scm.com/doc

Documentación oficial de Eclipse IDE: https://www.eclipse.org/documentation

Eclipse Temurin (Adoptium) — distribución del JDK 17: https://adoptium.net